

deep daiv. vol.0

PART

1

인공지능에 뛰어들 준비

1	COVER STORY: AI의 미래	8
2	인공지능 입문을 위한 필수 개념 총정리	12
3	AI와 함께 하는 딥다의 하루	26
4	지역별 AI 정책 훑어보기	40
5	INTERVIEW: 우리만이 할 수 있는 AI	52

PART

2

인공지능 세계에서 헤엄치기

1	INTERVIEW: AI 연구원의 길 - 대학원과 취업 사이의 선택	60
2	읽어봐요 AI의 숲	68
3	2024 AI Trend Keyword	74
4	INTERVIEW: 인공지능 대학원 탐방	90
5	INTERVIEW: 정부출연연구소의 세계로	96

PART

3

인공지능에 빠져들기

1	첫 코딩은 계획대로 될 거야!	106
2	OPINION: Apple Intelligence는 최고가 될 수 있을까?	120
3	세상을 바꾸는 AI 기업들	128
4	deep daiv into Kaggle & Dacon	146



인공지능 입문을 위한 필수 개념 총정리

에디터

저니

온 세상이 인공지능으로 떠들썩한 요즘, 인공지능을 한 번도 들어보지도 경험해 보지도 못한 사람은 없을 것이다. 본격적으로 인공지능의 세계에 빠지기 전에 뽀뽀이 흩어져 있는 필수적인 인공지능 개념들을 한 데 모아 정리했다. 이 글을 통해 인공지능을 처음 만나는 사람은 알아가는 시간을, 인공지능을 전에 접해본 사람은 정리하는 시간을 가졌으면 한다.

인공지능이라는 광활한 바다에 들어온 것을 환영하며, 깊게 숨을 들이쉬고 함께 **deep daiv!**

1. 프로그래밍 언어

프로그래밍 언어는 컴퓨터와 상호작용할 수 있는 수단이다. 논문을 읽는 것에서 나아가 인공지능을 더 심도있게 이해하고 직접 구현하기 위해 프로그래밍 언어를 아는 것은 필수적이다.

프랑스로 여행을 간 당신은 난처한 상황에 처해 현지인에게 질문을 하고 싶다. 하지만 당신이 불어를 못한다면 그들과 어떻게 의사소통할 것인가? 당신은 영어를 사용해 'Excuse me...'라고 할 것이다. 당신은 불어를, 프랑스인들은 한국어를 이해하지 못하기 때문에 모두가 알아들을 수 있도록 영어를 사용할 것이다.

프로그래밍 언어도 이 상황에서의 영어와 같은 맥락으로 이용된다. 인간과 컴퓨터가 모두 이해할 수 있는 언어를 사용해야 하는데, 그런 언어를 바로 '프로그래밍 언어'라고 부른다. 인간은 자연어*를 사용하고 컴퓨터는 0과 1로 이루어진 기계어*로 작동한다. 당연히 인간은 기계어를, 컴퓨터는 자연어를 곧바로 이해하지 못하기 때문에 프로그래밍 언어가 필요하다.

여기서 '프로그래밍'이란 무엇일까?

프로그래밍이란 인간이 생각하는 것을 컴퓨터에게 지시하는 행위를 의미한다. 즉, 프로그래밍 언어를 이용해 코드*를 짜서 컴퓨터에게 명령할 일을 정리해주는 것이다. 그래서 우리는 프로그래밍을 일상적으로 '코딩'이라고도 부르기도 한다.

* 자연어 사람들이 일상적으로 쓰는 언어

* 기계어 컴퓨터가 직접 해독하고 실행할 수 있는 비트 단위로 쓰인 언어

* 코드 Code 소스코드 Source Code의 줄임말로 프로그래밍 언어로 작성한 글을 의미한다.

프로그래밍 언어의 종류

영어, 한국어, 중국어, 일본어 등. 수많은 자연어가 존재하듯 프로그래밍 언어도 그 종류가 다양하다. 그중에서도 대표적인 파이썬 Python과 C언어 2가지를 비교하며 프로그래밍 언어의 특성을 알아보자.



- 객체 지향 언어*
- 인간 사고 체계와 유사
- 이해가 쉬움
- 동적으로 유형이 지정함



- 절차 지향 언어*
- 컴퓨터 사고 체계와 유사
- 상대적으로 성능이 우수
- 효율적인 메모리 사용량

파이썬은 인간이 직관적으로 이해하고 사용하기 좋지만, C언어는 연산 속도 관점에서 성능이 더 우수하다. 두 언어의 장점이 각각 명확하기 때문에 상황에 따라 다르게 사용한다.

* 객체 지향 언어 다수의 객체를 만들고, 객체 사이의 상호작용을 중점적으로 프로그래밍하는 언어

* 절차 지향 언어 코드를 최소화하여 최대한 빠르게 구현하는 것을 중점으로 프로그래밍하는 언어

인공지능과 파이썬

인공지능 분야에서 주로 사용되는 프로그래밍 언어는 파이썬이다.
그렇다면 파이썬이 왜 널리 쓰이는지, 그 이유를 알아보자.

(1) 단순하고 직관적인 구문

파이썬은 다른 언어에 비해 문법이 쉽고 간결하며 인간의 사고 체계와 유사하다는 장점이 있다. 위의 사진은 각각 파이썬과 C언어를 이용해 이름을 출력하는 코드이다. 얼핏 보아도 파이썬이 훨씬 간단하다는 것을 알 수 있다.

이러한 직관적인 구문은 직접 프로그래밍을 하는 경우에도 편리하지만, 다른 사람이 작업한 코드도 쉽게 이해할 수 있어 공동 작업이나 유지 보수가 편하다는 장점 또한 존재한다.



```
print("파이썬")
```



```
#include <stdio.h>

int main()
{
    printf("C언어");
    return 0;
}
```

(2) 다양한 라이브러리 활용

하지만 파이썬은 인간과 사고 체계가 비슷하기에 컴퓨터는 이해하기 어려워, 복잡하고 반복 연산이 많은 경우는 시간이 오래 걸린다. 이러한 단점은 라이브러리*와 프레임워크*로 극복할 수 있다.

파이썬은 라이브러리와 프레임워크의 활용도가 높은 언어이다. 다양한 언어로 만들어진 라이브러리를 이용해 파이썬이 가지고 있는 단점은 줄이고 장점을 부각시켜 더욱 많은 분야에서 활용할 수 있다.

예를 들어 프로그램의 빠대는 파이썬으로 만들고 빠른 실행이 필요한 라이브러리는 C언어로 만들어 파이썬 프로그램 안에 포함시킬 수 있다.

* **라이브러리 Library** 컴퓨터 프로그램에서 많이 사용되는 부분의 프로그램을 모은 집합

* **프레임워크 Framework** 특정한 종류의 개발을 하기 위한 기본 개념 구조. 프로그래밍 개발에 있어서 뼈대 역할을 한다.



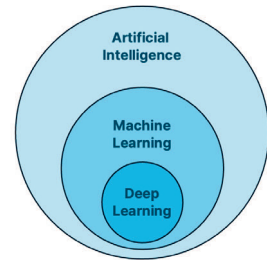
파이토치 PyTorch 도 인공지능 분야에서 파이썬을 통해 널리 사용하는 하나의 프레임워크다. 파이토치에 대한 내용은 part.3에서 더 확인해볼 수 있다.

2. 인공지능과 머신러닝 그리고 딥러닝

인공지능 & 머신러닝 & 딥러닝의 관계

거대한 인공지능이라는 분야 속 인공지능을 학습하는 하나의 방법이 **머신러닝**이다. 그러한 머신러닝 중에서 인공 신경망을 이용해 학습하는 방법을 **딥러닝**이라고 부른다. 우리가 현재 열광하고 있는 인공지능은 인공지능의 분야 속에서도 딥러닝 방법론이며, 딥러닝은 인공지능 기술 발전의 꽃을 피우고 현재 인공지능 시대를 연 장본인이다.

그렇다면 인공지능은 과연 무엇인지, 머신러닝과 딥러닝을 함께 짚어보면서 인공지능 속 다양한 개념에 대해 살펴보자.



인공지능의 분류

2.1. 인공지능 Artificial Intelligence, AI

인공지능은 인간이 학습하고 추론하고 의사 결정하는 방식을 모방하거나, 인간을 넘어서는 지능을 기계에서 구현하는 기술이다.

인공지능의 개념은 광범위하기 때문에 관점에 따라 다양하게 분류할 수 있다.

인간의 관점에서 시스템의 의식 수준에 따라 강한 인공지능 ^{Strong AI}와 약한 인공지능 ^{Weak AI}으로 나눈다. 기능적 측면에서 강한 인공지능은 인간의 모든 지능 기능을 본뜬 것이며, 약한 인공지능은 특정 지능에 제한된 작업을 수행할 수 있는 것이다. 현재는 약한 인공지능 수준의 모델만 나오고 있다.

2.2 머신러닝 Machine Learning

인공지능에서 컴퓨터는 작업을 어떻게 수행할까? 기존 프로그래밍에서는 인간이 규칙을 직접 지시했다면, 그 틀을 깬 학습 방법이 바로 머신러닝이다. 이에 대해 자세히 알아보자.

머신러닝이란?

컴퓨터 시스템이 데이터를 스스로 학습하고 결정내릴 수 있도록 훈련하는 인공지능의 한 방법론이다. 입력한 데이터에서 컴퓨터의 알고리즘*이 패턴이나 특징을 인식하고 새로운 문제를 해결한다.

* 알고리즘 Algorithm

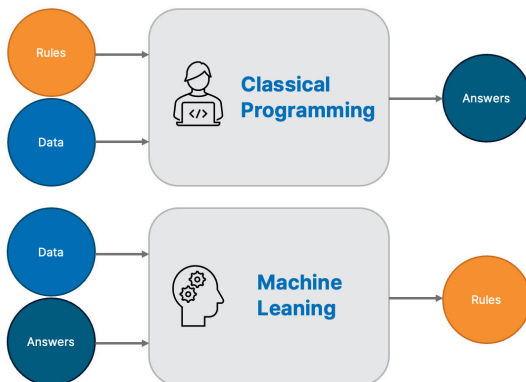
특정 문제를 해결하기 위한 단계적인 절차나 규칙

기존의 프로그래밍 vs. 머신러닝

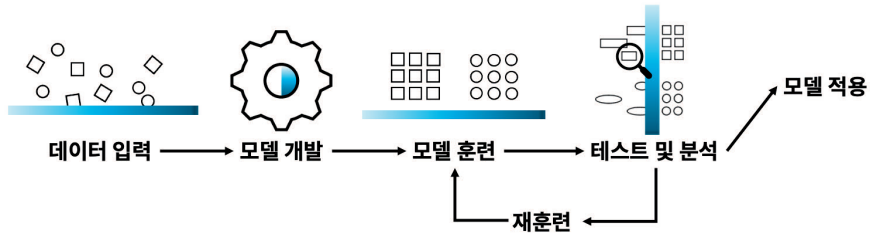
기존의 프로그래밍은 입력 데이터에 문제 해결을 위한 명시적인 알고리즘을 추가하면 결과값이 나오게 된다.

머신러닝은 데이터 사이의 패턴이나 관계를 학습하여 모델을 만들어낸다.

이로 인해 다양한 차이점이 발생하는데, 기존의 프로그래밍은 데이터의 양과 질에 큰 영향을 받지 않지만, 머신러닝은 매우 중요하다. 그러나 알고리즘이 복잡할수록 기존의 프로그래밍으로 개발하기 어려워진다. 정리하자면, 규칙이나 특징이 단조로우면 기존의 프로그래밍이 빠르고 효율적이겠지만, 문제가 복잡하고 정밀할수록 머신러닝을 이용한 인공지능이 더욱 강점을 발휘할 것이다.



머신러닝의 과정



(1) 데이터 입력

데이터를 수집하고 전처리*를 거치는 단계

(2) 모델 개발

구현하기 위한 작업과 기능에 맞는 학습 모델을 선택하고 개발하는 단계

(3) 모델 훈련

학습용 데이터와 검증용 데이터를 이용해 인공지능을 구현하는 단계

(4) 테스트 및 분석

학습이 종료된 모델을 평가용 데이터를 이용해 품질을 평가하는 단계

(5) 모델 적용

실제 환경에 예측할 수 있는 환경을 만들고, 실제 환경에 적용할 수 있도록 배포하는 단계

* 데이터 전처리 Data Preprocessing 수집한 원시 데이터(Raw Data)를 머신러닝에 더 적합한 형태로 변환하는 단계



데이터의 종류

- 정형 데이터: 명확한 구조를 가지고, 일반적으로 표 형식으로 저장되는 데이터
ex) csv 등
- 반정형 데이터: 명확한 구조는 없어도, 특정한 규칙에 의하여 작성된 형태의 데이터
ex) XML, JSON 등
- 비정형 데이터: 일정한 형식이 없거나 데이터 자체가 형식인 데이터
ex) 음성, 이미지, 동영상 등

데이터 전처리의 목적

- 데이터 품질 개선: 노이즈, 이상치 등을 제거해 신뢰할 수 있는 데이터를 만든다.
- 데이터 표준화: 수집된 데이터의 형식이나 척도의 차이를 맞춘다.
- 특성 추출: 적절한 특성을 추출 혹은 생성한다.
- 과적합* 방지: 불필요한 정보를 제거해 훈련 데이터에 지나치게 의존하는 것을 막는다.

* 과적합 Overfitting 기계 학습에서 학습 데이터를 수집할 때 전체적인 경향성에서 벗어난 데이터까지를 고려하여 실제 데이터에 대해서는 일반화 성능이 떨어지는 모델을 얻게 되는 현상. 정해진 데이터만으로 너무 많은 학습을 하면 생길 수 있다.

데이터의 분할 및 훈련과정

가공한 데이터를 모두 학습에 사용하는 것은 아니다.

학습-검증-테스트의 훈련 과정에 대응해 학습용, 검증용, 테스트용 데이터로 분할해서 사용한다.



학습 Training

모델이 데이터에서 패턴을 학습하고 일반화하는 용도로 사용된다. 제일 많은 양의 데이터가 필요하다.

검증 Validation

모델이 올바르게 학습되고 있는지 확인하기 위해 사용된다. 모델의 학습 방향성을 지정한다.

테스트 Testing

이 모델이 얼마나 잘 학습된 것인지 확인하는 데에 사용된다. 학습한 모델에 테스트 데이터를 넣어 원하는 답과 비교해보면서 성능을 확인한다.

모델의 훈련 과정을 수능을 치는 수험생에 비유해보겠다. 수능을 위해 공부를 하는 것이 학습 Training 과정이다.

모의고사를 통해서 내가 올바르게 공부를 하고 있는지, 어느 부분이 부족한지 확인하는 것이 검증 Validation 과정이라고 보면 되겠다. 테스트 Testing 과정은 수능을 치고 결과가 나오는 것이라 할 수 있다.

머신러닝의 학습 방식

머신러닝은 학습 모델의 기능과 방법에 따라 다르게 분류할 수 있다.

지도학습 - 비지도학습으로 나뉘 보았다. 각각의 학습 방법에 대해 알아보자.

Supervised Learning vs Unsupervised Learning

(1) 지도 학습



출처: freeCodeCamp

위 사진에서 치와와 사진은 총 몇 개일까? 사람은 쉽게 분류할 수 있지만 인공지능에게는 이야기가 다르다. 하지만 이 중에서 어떤 사진이 '치와와'이고 '머핀'인지 미리 AI에게 알려준다면, '치와와 & 머핀 사진 분류' 모델을 만들 수 있다. 여기서 사용된 학습 방법이 바로 **지도 학습**이다.

지도 학습이란, 출력할 데이터인 레이블 Label 을 정의하여 입력 데이터와 함께 학습을 하는 방법이다. 위의 사례에서는 입력 이미지와 함께, 그 이미지가 치와와인지 머핀인지 알려주는 레이블이 학습에 사용된다. 이러한 과정을 거치면, 모델이 치와와와 머핀 이미지의 관계를 학습하여 새로운 사진에 대해서도 예측을 할 수 있다.

지도 학습의 예시

① 회귀 Regression

회귀는 연속값을 예측하는 작업이다. 입력 데이터에 기반해 수치적 출력값을 예측한다. 예를 들어 주택의 크기나 위치, 연식 등 주택에 대한 다양한 정보로 주택의 가격을 예측하고자 할 때 사용할 수 있는 모델이다.

② 분류 Classification

분류는 특정한 범주나 클래스에 입력값을 할당하는 작업이다. 즉, 주어진 입력에 대해 정의된 클래스 중 하나를 선택하는 것이다. 로지스틱 회귀도 분류의 클래스가 2개인 분류 작업이다.

자가 지도 학습 Self-Supervised Learning, SSL

자가 지도 학습은 레이블이 없는 데이터를 레이블이 부여된 데이터처럼 바꿔 지도 학습을 하는 방법이다. 레이블이 없는 데이터에 레이블을 부여하는 방법으로는 대표적으로 마스킹*이 있다. 마스킹을 통해 가려진 부분을 정답(레이블)로 생각해 처리할 수 있다.

* 마스킹 Masking 데이터의 일부를 랜덤하게 제거하거나 가리는 기법

(2) 비지도 학습

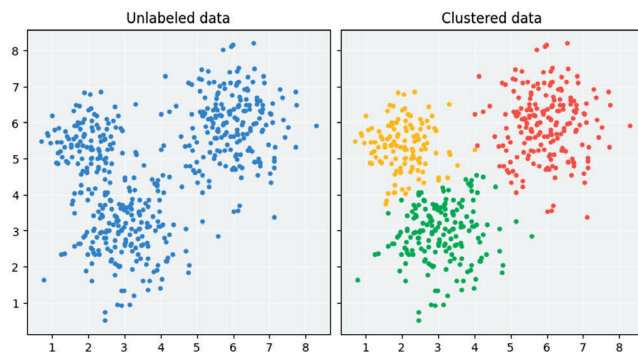
만약 당신이 대형 백화점을 운영하고 있다면, 고객의 특성을 파악하는 것은 중요한 과제일 것이다. 여러 특성을 바탕으로 고객을 적절히 그룹화하는 것은 마케팅 전략에 있어 중요한 지점이며, 이는 이윤 창출과 직결되기 때문이다. 구매 패턴이나 선호도 등의 데이터를 활용해 고객을 여러 그룹으로 나누기 위해서는 모델을 어떻게 학습시켜야 할까? 이러한 경우 적합한 방식이 바로 **비지도 학습**이다.

비지도 학습은 지도 학습과 달리 명시적인 레이블 없이 모델을 훈련하는 학습 방법이다. 주어진 데이터에서 패턴을 파악하여 특정 작업을 수행한다. 위의 예시는 다양한 데이터를 이용해 정답이 정해져 있지 않는 그룹을 만드는 작업인 것이다.

비지도 학습의 예시

① 군집화 Clustering

비슷한 특성을 갖는 데이터를 묶는 작업이다. 유사성을 기준으로 데이터의 구조를 파악하고 분류하는 데 도움이 된다.



출처: Google for Developers

② 차원 축소 Dimension Reduction

고차원의 데이터를 저차원으로 변환하는 작업이다. 차원을 줄인다는 의미는 데이터가 가지고 있는 특성의 수를 줄여 불필요한 정보를 제거한다는 뜻이다. 이는 특성 추출 과정으로도 볼 수 있다. 차원 축소 과정은 데이터의 시각화, 노이즈* 제거, 효율성 향상 등에 많은 도움을 줄 수 있다.

③ 이상치(특이치) 탐지 Anomaly Detection

데이터에서 발견되는 일반적인 패턴과 다른, 예상치 못한 사건을 탐지하는 기술을 의미한다.

* 노이즈

잡음이라는 뜻처럼, 예측할 수 있는 패턴에서 벗어나는 데이터 혹은 데이터의 일부

지도 학습 VS 비지도 학습

지도 학습은 데이터가 정확히 예측해야 하는 정답 값을 포함하고 있어 모델의 성능 평가가 가능하고, 이에 따라 효율적인 학습이 가능하다. 다만, 대량의 데이터에 레이블을 다는 작업이 필요하기 때문에 양질의 데이터를 구축하는 데 많이 비용이 든다는 단점이 있다.

비지도 학습은 레이블이 필요 없기 때문에 학습에 있어 많은 데이터를 쉽고 경제적으로 이용할 수 있다. 하지만 지도 학습만큼 빠르고 효율적인 학습은 어려울 수 있다. 또한 레이블이 없어 학습 모델을 평가하거나 검증하기 쉽지 않다.

	지도	비지도
데이터	입력 데이터와 레이블	입력 데이터
목표	주어진 입력에 대한 올바른 레이블 예측	데이터의 패턴이나 구조 발견
학습 방식	입력-출력 매핑 학습	데이터의 구조 학습
손실 함수*	예측 오류를 최소화하는 손실 함수 사용	거리 최소화, 분산 최대화 등 기준 사용

* 손실 함수 Loss Function 실제 결과값과 모델의 예측값의 차이를 최소화하도록 정의된 특수한 함수

(3) 강화 학습

강화 학습은 어떤 행동에 대해 보상과 페널티를 부여함으로써, 주어진 환경에서 최적의 행동을 할 수 있도록 하는 학습 방법이다.

강화 학습에 필요한 개념은 다음과 같다.

- **에이전트** Agent : 의사결정을 내려 행동하는 주체
- **환경** Environment : 에이전트가 상호작용하는 외부 시스템
- **상태** State : 에이전트가 하는 행동에 따라 상태는 변하는 에이전트가 현재 속한 상태
- **행동** Action : 에이전트가 수행한 행동
- **보상** Reward : 에이전트가 행동을 했을 때, 환경으로부터 받는 피드백
- **가치** Value : 장기적인 측면에서의 보상의 총 기대값

결국 강화 학습은 에이전트가 주어진 환경에서 행동에 따른 보상의 구조를 학습하고, 가치를 최대화하는 최적의 행동 방향을 찾는 것을 목표로 한다.

강화 학습의 예시로는 2016년과 2017년에 세상을 떠들썩하게 한 알파고^{AlphaGo} 프로그램이 있다. 알파고는 수백만 개의 게임을 강화 학습 방법론으로 학습했다.

2.3 딥러닝 Deep Learning

방수복, 낙하산, 벨크로, 그리고 딥러닝. 이들의 공통점은 무엇일까? 바로 자연으로부터 영감을 얻어 개발됐다는 것이다. 방수복은 연잎에서, 낙하산은 민들레 씨에서, 벨크로는 도꼬마리 가시에서 아이디어를 얻어 탄생했다.

머신러닝 중에서도 인간의 신경세포로부터 영감을 받아 만든 학습 방법이 있다. 이를 **딥러닝**이라고 부른다.

딥러닝이란?

머신러닝의 여러 알고리즘 중 인공 신경망을 기반으로 한 방법을 통칭하여 말한다. 복잡한 패턴을 학습하는 데 중점을 둔다.

일반 머신러닝

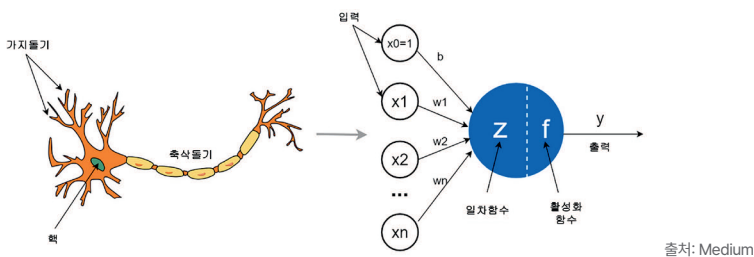
- 간단한 모델로도 성능을 낼 수 있음
- 상대적으로 적은 자원이 필요함
- 일반적으로 적은 데이터로도 일정 수준의 성능을 가짐
- 복잡한 문제에는 한계가 있음

V
S

딥러닝

- 대량의 데이터와 깊고 복잡한 모델이 필요함
- 많은 계산 자원과 시간이 필요함.
- 대규모 데이터와 높은 복잡성으로 뛰어난 성능을 보임
- 적은 데이터로는 과적합 위험이 높음

이때 인공 신경망 Artificial Neural Network, ANN 이란, 생물학적 신경망에서 영감을 받아 구축된 컴퓨터 알고리즘이다.



인공 신경망은 사람의 신경세포인 뉴런과 작동 방식이 비슷하다.

뉴런은 신경세포로 전달되는 여러 자극값의 합이 일정 기준을 넘긴다면 다음 세포로 전달되고 아니면 소멸되는 방식으로 작동한다. 초기 인공 신경망인 퍼셉트론* 역시, 각각의 가중치*를 갖는 변수들을 합해 일정 수준 이상이 되면 출력을 활성화하고, 아니면 비활성화하는 형태이다.

1차식으로 간단하게 표현하였지만, 가중치나 활성화 함수*도 이용하여 뉴런의 복잡한 메커니즘을 수식으로 표현했다는 의의를 가지고 있다.

* **퍼셉트론 Perceptron** 딥러닝의 근간으로, 1957년에 로젠블라트가 고안한 가장 간단한 형태의 네트워크.

* **가중치 Weight** 각 연결의 중요도를 의미하는 데이터를 처리하는 중요 매개변수 중 하나로, 입력 값에 가중치가 곱해져 다음 층으로 전달된다.

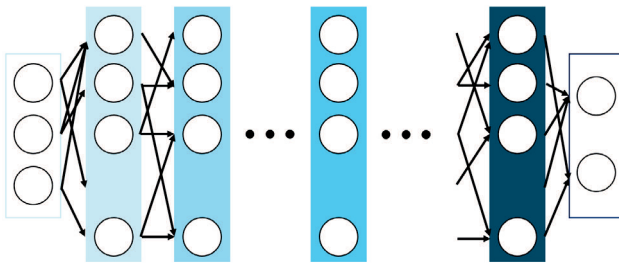
* **활성화 함수 Activation Function** 인공 신경망에서 입력 신호를 받아 출력으로 변환하는 함수. 다양한 활성화 함수가 있지만, 어떤 활성화 함수를 선택하느냐에 따라 신경망의 성능과 학습 능력에 큰 영향을 미친다.

딥러닝 모델의 종류

딥러닝에서 사용하는 인공 신경망은 퍼셉트론 이후로도 많은 발전을 이루었고, 이를 활용한 다양한 모델이 등장했다. 여기서 모든 모델을 다룰 수는 없지만, 인공지능을 알기 위해 필수적인 몇 가지 모델을 알아보고 모델의 구성 방식을 살펴보자.

(1) DNN Deep Neural Network

DNN은 입력층*과 출력층* 사이에 은닉층*을 2개 이상 갖는 인공 신경망이다.



- * 층 Layer 인공 신경망의 구조적 깊이를 의미하며, 층의 종류에는 입력층, 출력층, 은닉층이 있다.
- * 입력층 문제와 관련된 데이터를 입력받는 층
- * 출력층 모델이 결과를 산출하는 층
- * 은닉층 입력층과 출력층 사이에 위치하며, 데이터의 특성을 보다 세밀하게 반영할 수 있도록 가중치를 복잡하게 구성하는 층
- * 노드 Node 신경망에서 데이터를 저장하고 갱신하는 단위, 셀이나 뉴런으로도 불린다.

즉 더 많은 층*이 생기고 노드*의 개수가 많아지면서, 데이터를 더욱 깊고 넓게 처리할 수 있게 됐다. 그래서 복잡한 패턴과 특성을 학습하는데 매우 효과적이라는 장점이 있다.

DNN의 등장이 가져온 나비효과

기존의 프로그래밍은 개발자의 정밀한 논리에 의해 짜인 순서대로 작동되었기 때문에, 단계별로 모델에서 어떤 일이 일어나는지 알 수 있었다.

그러나 DNN 등장 이후, 모델은 우리의 상상을 넘어서는 연산을 진행하게 되었다. 모델이 복잡해지면 개발자조차도 모델 속 파라미터*가 얼마인지, 중간 과정이 어떻게 진행되는지 알거나 이해하기 힘들어진다. 따라서 딥러닝 모델은 기존의 프로그래밍처럼 ‘정밀하고 치밀할’ 이유가 없으며, 이를 해석하는 인간도 경험과 결과에 의한 선택을 하게 되는 것이다.

이는 프로그래밍의 개념이 기존의 ‘논리적’ 접근 방식에서 ‘실험적’ 접근 방식으로 확장되었다는 것을 의미한다. DNN의 등장은 프로그래밍에서 획기적인 패러다임의 변화를 불러오게 되었다.

* 파라미터 Parameter 모델이 학습을 통해 조정하는 값. 대표적인 파라미터로는 가중치나 편향*이 있다.

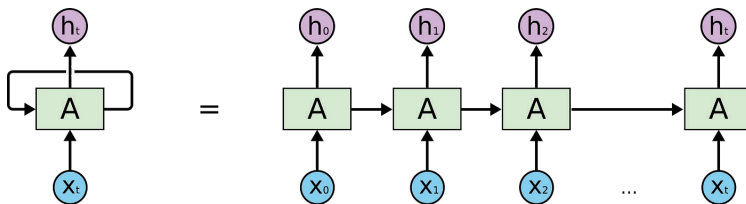
* 편향 Bias 각 노드에 추가되는 상수항. 가중치는 입력값에 곱해서 중요도를 표현한다면, 편향은 더해서 출력값을 유연하게 만들어준다.

DNN은 딥러닝의 기본적인 모델이다. DNN의 등장으로 머신러닝의 학습 능력은 폭발적으로 증가하기 시작했다. DNN을 응용한 여러 모델을 아래에서 알아보자.

(2) RNN Recurrent Neural Network

‘동해물과 ____이’ 에서 빈칸에 어떤 말이 들어갈까? 당신은 아마 ‘백두산’을 떠올렸을 것이다. 문장 속 빈칸을 채울 때는 구조상 앞 뒤 단어나 맥락이 중요하게 작용한다. 이러한 방식으로 앞뒤 데이터의 연관성을 학습하고 그 연관성을 고려한 예측을 수행하는 모델이 RNN이다.

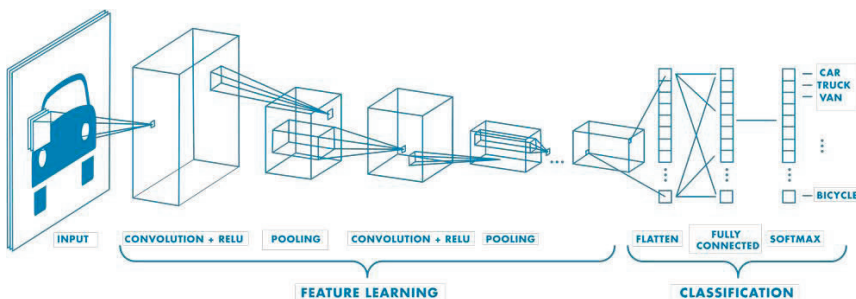
RNN Recurrent Neural Network 은 DNN의 일종으로, 시간적 혹은 공간적 연속성이 있는 데이터를 다루는데 적합한 모델이다. RNN은 'Recurrent'라는 이름에 걸맞게 순환하는 구조를 가지고 있어 이전의 정보를 기억하고 현재의 입력에 대한 출력을 생성한다.



출처: Medium

RNN에 장점만 있는 것은 아니다. 그림을 보면 RNN은 하나의 연산 결과를 계속 덮어 쓰는 방식으로 다음 셀에 넘기게 된다. 그래서 입력값이 앞 부분에 위치할수록 학습 과정에서 정보가 소실될 위험이 있다. 이를 장기의존성 문제라 부른다.

(3) CNN Convolutional Neural Network



출처: MathWorks

RNN에서 문장형 데이터를 다뤘다면, 데이터가 이미지 형태인 경우는 어떻게 처리할 수 있을까? 이미지는 인접한 픽셀 간 관계가 중요하기 때문에 이에 대한 특별한 처리를 해준다면 모델이 효과적으로 학습할 수 있을 것이다. CNN이 그러하다.

CNN 역시 DNN의 일종이다. 데이터 처리에 적합하도록 행렬로 표현된 필터를 사용해 필터링을 하는, 시각적인 효과를 처리할 수 있는 모델이다. 가중치를 하나의 면 형태로 구성하는데, 이를 통해 이미지의 한 화소가 주변 화소와 맺는 관계나 특징을 파악할 수 있다. CNN은 여러 개의 층으로 구성되어 이미지의 특징을 추출하고, 분류하고, 예측을 진행한다.

합성곱 층 Convolution Layer

CNN에서 핵심적인 역할을 하는 층으로, 이미지에서 지역적인 특징을 감지하기 위해 필터(커널*)를 사용하여 합성곱 연산을 진행한다. 이 과정에서 활성화 함수를 적용해 특성 맵*을 생성한다.

풀링 층 Pooling Layer

합성곱 층에서 생성된 특징 맵의 크기와 계산량을 줄이기 위한 층으로, 맥스 풀링 Max Pooling 이나 평균 풀링 Average Pooling을 사용하여 각 영역의 최대값이나 평균값을 추출한다.

스트라이드 Stride

필터가 이동할 때, 이동 간격을 결정하는 값이다. 스트라이드 값이 작으면 겹침이 많고, 크면 겹침이 적어져 특징 맵 크기를 조절할 수 있다.

완전 연결 층 Fully Connected Layer

모든 입력 뉴런이 다음 층의 뉴런과 연결되는 층으로, 합성곱 층과 풀링 층을 통해 추출된 특징을 기반으로 분류를 수행한다.

* 커널 Kernel 합성곱 연산을 진행하는 작은 행렬로, '필터'로도 불린다. 이미지에 대해 이동하면서 합성곱을 진행해 특징을 추출한다.

* 특성 맵 Feature Map 합성곱 연산 후 생성되는 결과. 입력한 데이터의 특성이나 패턴을 강조한 맵을 의미한다.

개선된 다른 딥러닝 모델들

1. 트랜스포머 Transformer

2017년 <Attention is all you need> 논문에서 소개된, 자연어 처리 및 기계 번역 등 인공지능 작업에서 사용되는 혁신적인 딥러닝 아키텍처이다. 셀프 어텐션 메커니즘*을 사용하여 문장의 각 단어가 다른 단어들과 어떤 연관을 가지는지 파악하고 RNN의 장기 의존성 문제를 해결하는 큰 발전을 가져왔다.

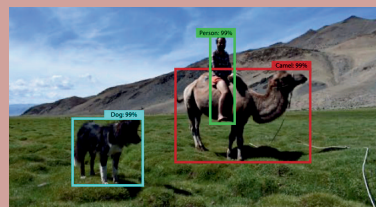
* 셀프 어텐션 메커니즘 Self-Attention Mechanism 주어진 시퀀스 내 단어가 다른 단어와 얼마나 관련되어 있는지 계산하는 방식

* 그리드 Grid 이미지 내의 객체를 탐지하기 위해 이미지 공간을 나누는 방법

* 바운딩 박스 Bounding Box 이미지 내의 특정 객체의 위치나 크기를 정의하는 직사각형

2. Yolo You Only Look Once

2015년에 나온 객체 탐지를 위한 딥러닝 알고리즘이다. Yolo는 번거로웠던 CNN의 과정을 제거하고자, 이미지를 그리드*로 나누고, 각 그리드 셀에서 객체의 바운딩 박스*와 클래스를 확률로 예측하는 과정을 가진다. 이로써 이미지를 여러 부분으로 나누는 대신에 전체 이미지에 대한 예측을 한 번에 진행하기 때문에 속도가 빠르고 실시간 객체 탐지에 효과적이다.



많은 내용을 한 번에 다뤘기 때문에 곧바로 이해하지 못하더라도, 차근차근 읽고 궁금한 내용을 스스로 공부하다 보면 어느새 인공지능의 바다를 유유히 떠다니는 당신을 볼 수 있을 것이다.

NAVER

여러 기사를 통해 우리는 많은 기업이 인공지능 분야에 많은 관심을 기울이고 있다는 것을 알 수 있다. 또 기업의 다양한 서비스를 사용하며 인공지능이 삶 깊숙이 들어와있음을 몸소 느낄 수도 있다. 그렇다면 기업은 인공지능을 어떻게 바라보고 있을까? 국내 최고의 AI 기업 중 하나인 네이버의 이야기를 네이버 책임리더 Executive Officer의 생각과 함께 들어보자.

INTERVIEW : 우리만이 할 수 있는 AI

에디터 저니



안녕하세요, 강인호입니다.

네이버에 2008년에 입사하여 17년째 검색_{Search} 분야에서 계속 연구를 진행하고 있습니다.

예전에는 텍스트의 형태로 특정한 '문서'만 검색할 수 있었습니다. 기술이 발전함에 따라 음성이나 사진으로 검색하는 등 다양한 활동을 할 수 있게 되었습니다. 최근에는 LLM과 생성형 AI를 검색 서비스와 접목하는 연구도 하고 있습니다.

진행한 연구 중 기억에 남는 연구가 있으신가요?

스마트 스피커 연구가 가장 기억에 남습니다. 저는 박사학위도, 박사 후 과정_{Postdoctoral Researcher}도 '검색' 분야로 연구했습니다. 대개 직업은 전공과 무관한 경우가 많은데, 20년 넘게 공부한 분야를 업으로 삼을 수 있는 운 좋은 사람이기도 하죠.

과거에는 컴퓨터를 활용해 검색했지만 이제는 휴대전화에서 검색합니다. 미래의 검색은 운전하거나 걸어가면서 언제든지 사용할 수 있는 음성을 활용한 형태라고 예상했습니다. 그래서 스피커를 연구하면서 이 서비스가 큰 변화를 불러올 것이라 기대했죠.

하지만 지금 생각하면 기술이 완성되지 않은 상황에서 선부르게 서비스를 시도한 것 같습니다. 그리고 한국의 문화와 접목하기 쉽지 않았습다. 언제든지 자연스럽게 사용할 수 있길 기대했지만, 지하철 안이나 사무실에서 말한다는 것이 익숙하지 않은 문화였죠. 그런 몇 가지 부분이 조금 아쉬웠지만 그래서 더 기억에 남는 연구인 것 같습니다.



네이버와 인공지능

회사에서 인공지능을 사용하면서 변한 점은 어떤 것이 있나요?

개발 주기가 짧아졌습니다. 예전에는 하나의 서비스를 개발하기 위해 많은 노력이 필요했습니다. 학습 데이터셋 구축, 모델 구현, 테스트까지 시간이 몇 달 걸리기도 했죠. 하지만 인공지능을 사용하면 며칠 안에 품질 개선 과정이 끝나기도 합니다.

또한 데이터셋 구축을 위해 여러 사람을 모으는 것 자체가 쉽지 않았는데, 인공지능을 통해 어느 정도 사람의 역할을 대체하는 것이 예전보다는 도움이 되고 편해서 좋습니다. 예를 들어 사람들에게 직접 설문지를 주지 않고도, 인공지능 모델에 캐릭터나 페르소나를 주고 '너는 20대 사회 초년생인데 이런 문제는 어떻게 생각해?' 같은 질문을 던져 예상 답안을 얻을 수 있습니다.

일을 효율적으로 만들어 준다는 점도 있습니다. 예컨대, 인공지능이 초안 수준으로 번역해주면 이를 약간만 고쳐 사용할 수 있습니다. 또 논문을 읽을 때, 제가 관심이 있는 논문인지 일차적으로 걸러주기만 해도 편하게 고를 수 있죠.

네이버에서는 인공지능을 어떻게 사용하고 있나요?

개발자가 많이 필요한 일이나 데이터 어노테이터^{Annotation}들이 품질 검사하던 일에 적극 사용합니다. 물론 제가 하는 검색 분야 외에 클로바^{CLOVA}*에서 인공지능 모델 자체를 만드는 것도 집중하고 있습니다. 나아가 저희의 모델을 통해 새로운 사업을 만들거나 클라우드를 지원하면서 인공지능의 개발 환경에 긍정적인 영향을 줄 수 있다고 기대합니다.

* CLOVA

네이버에서 개발한 인공지능 플랫폼.
HyperCLOVA X처럼 네이버의 생성형 AI 모델도 있고, 문서요약, 상품추천, 챗봇 등등 인공지능을 이용한 다양한 서비스도 사용할 수 있다.

네이버는 다양한 분야를 시도합니다. 현재 한국에서 생성형 AI 모델을 만들기도 하고, 특히 사용자가 원하는 것을 도와준다는 관점으로 인공지능 서비스에 집중합니다. 사용자가 원하는 콘텐츠를 빨리 만들 수 있게, 또한 어려웠던 것을 쉽게 접근할 수 있게 하는 데에 생성형 AI를 쓰려고 노력합니다.

그래도 가지고 있는 고민이 있다면 어떤 점이 있을까요?

AI가 가진 잠재력을 악용하는 사례에 있어서 걱정되는 부분이 있습니다. 개발자가 기획한 의도와 다르게 사용할 수 있다는 것이죠. 그러한 관점에서, 인공지능을 잘못 사용하면 어떤 결과가 나올지 모른다는 점이 우려되기도 합니다. 그러나 또 개발자이기 전에 한 개인으로서 기술 발전을 마냥 막으면 또 아쉽다고 느껴져 고민이 있습니다.

이와 관련해 인상 깊었던 이야기가 떠오르네요. ‘자동차는 개발자가 만들었지만, 신호등과 도로는 사회가 만들었다.’ 신호등과 도로가 만들어지면서 자동차 사고가 줄고 사회가 자동차를 받아들일 수 있었습니다. 이처럼 인공지능도 마찬가지로 될 것 같습니다.

물론, 생성 AI는 누구나 쉽게 활용할 수 있다는 점에서 자동차와는 차이가 있습니다. 때문에 오히려 더욱 세심한 배려와 관심이 필요하다고 생각합니다.

경기장과 경기 규칙만 있으면 저희는 열심히 따라갈 의지가 넘칩니다. 사회적으로 뒷받침된다면 개발하는 입장에서 더욱 박차를 가할 수 있다는 개인적인 기대감을 품고 있습니다

네이버는 인공지능에 대해 어떤 목적을 가지고 있나요?

제가 네이버를 대표할 수는 없지만 제가 저희 팀원에게 강조하는 이야기는 있습니다.

LLM을 연구하는 논문도 되게 많다. 기술도 다양하다. 하지만 여러분들이 자신 있게 말할 수 있는 것은 4천만 사용자를 대상으로 우리나라 서비스를 가지고 있다는 것이다.

가장 성능이 좋은 모델, 품질이 좋은 서비스만을 추구한다면 세상에 금메달 따는 사람 한 명만 있는 것이지 않습니까? 그렇다면 1등을 제외하면 나머지는 필요가 없는 걸까요? 저는 그렇게 생각하지 않습니다.

월마트가 글로벌하게 더 많은 제품과 더 싸게 줄 수 있지만 그 동네 슈퍼마켓만이 할 수 있는 것이 있습니다. 슈퍼마켓은 10년, 20년 넘게 동네 사람들을 잘 알기 때문에 사람들이 무엇을 원하는지 잘 알고 있습니다. 월마트에서 주지 못한 신선한 제품을 줄 수 있지 않을까요?

사람들을 만족시킬 수 있는 새로운 기술을 개발한다면 저희만의 차별점이 될 것이라 믿습니다.

저희도 세계적인 트렌드나 최신 기술을 수용하면서도 어떻게 우리 서비스에 적용해야 품질과 성능이 개선되는지 살펴봅니다. 자체적으로 보유한 기술을 지속적으로 개발해, 국내에서 제일 유용한 기술을 가진 회사가 되면 좋겠다고 생각합니다. ‘네이버니까! 네이버마저 안 하면 누가 할까? 만들어봐야지!’라고 생각하고 책임감을 가지고 있습니다.

리더님이 생각하시는, 인공지능을 사용하는 미래의 네이버는 어떤 모습일까요?

생성형 AI가 발전함에 따라서 에이전트 개념이 강화되고, 1인 사업자가 많이 나올 것이라 예상합니다.

에이전트를 활용함으로써 몇백 명의 일을 1명이 하는 것이 가능하다고 생각합니다. 그로 인해 점차 작은 규모로 사업이 더 많은 일들을 하는 식으로 바뀔 것 같습니다. 의사소통이나 업무 관리가 사업의 힘든 부분 중 하나입니다. 그렇기에 필요한 최소 인원만 고용하고 나머지는 에이전트나 로봇이 대체하는 사업 트렌드가 나올 수 있다 봅니다. 그리고 이러한 트렌드에 발맞춰 네이버도 도울 수 있지 않을까요?

네이버 쇼핑라이브* 로 예를 들 수 있을 것 같습니다. 홈쇼핑 같은 경우에 제품과 내용만 다르지, 촬영 구도가 비슷합니다. 현재는 스튜디오 대여나 인테리어를 위해 비용을 주고 수많은 사람이 붙어서 진행해야 합니다.

인공지능을 이용하면 프로프트를 바꾸는 것만으로도 쉽게 사용할 수 있을 거라 예상합니다. 이런 서비스를 만들어 네이버에서 제공해 준다면 보다 편리하게 활용하고 그에 맞는 플랫폼을 제작해 사용자들도 편하게 이용할 수 있다고 생각합니다.

네이버 내에서도 인공지능의 사용도 생각해 볼 수 있을 것 같아요. 개발자들이 가장 많이 하는 작업은 단연 코딩입니다. 병원에서 의사가 실수하면 안 되듯이, 저희도 코드 구현에 있어 실수하면 안 됩니다. 조건문 하나라도 서버가 종료될 수 있기 때문이죠. 그래서 페어 프로그래밍*이 개발자 사이에서 매우 중요한 과정입니다.

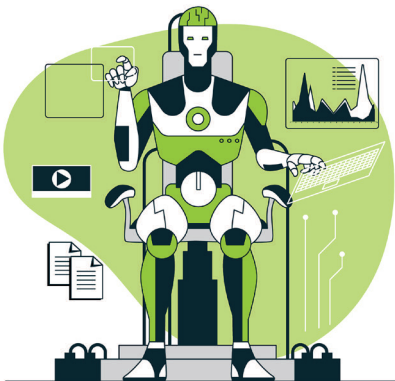
하지만 앞으로는 AI를 이용해 코멘트나 주석을 달기도 편하고, 변수명도 잘 맞춰 쉽게 오류를 잡을 수 있다면 전체적인 일의 능률이 향상될 것이라 예상합니다.

* 네이버 쇼핑라이브

쉽고 편리하게 라이브 방송을 촬영하여, 실시간으로 소통과 판매가 가능한 네이버의 플랫폼.

* 페어 프로그래밍

코드의 품질을 높이고 오류를 줄이기 위해 2명의 개발자가 하나의 작업을 함께 수행하는 코딩 방법.



인공지능을 접하는 독자에게 하고 싶은 조언이 있다면요?

최근에 부족 인공지능이 사람들에게 인기가 있다는 것을 느낍니다.

네이버에서 개최한 인공지능 대회에서 대학생이 1등을 했습니다. 예전에 인공지능은 석사나 박사만 공부했지만, 이제는 학부생, 심지어 고등학생도 높은 열정을 보이기도 합니다. 감사하게도 많은 사람이 관심을 가져서 빠르게 발전할 수 있다고 생각합니다.

하지만 지금까지의 인공지능은 사람이 못하는 창의적인 일을 할 수 있는 단계는 아닙니다. 그럼에도 인공지능을 사용하는 큰 이점은 효율성입니다. 빠르면서 실수도 줄여주기 때문입니다. 이것만으로도 전체적인 사회가 발전할 수 있다고 기대합니다.

반면 인공지능에 대한 의존성이 커질까 봐 우려하시는 분들도 많을 것으로 생각합니다.

핸드폰 연락처에 전화번호를 저장하면서 잘 외우지 않게 되었습니다. 하지만 전화번호를 모른다고 저희가 바보가 되었다고, 또 휴대전화에 의존적이라 얘기할 수 있을까요? 인공지능도 마찬가지입니다. 2시간 걸리는 일을 1시간 만에 할 수 있다면 아끼는 시간을 통해 다른 무언가를 할 수 있기 때문에 저는 그에 대한 가치를 높게 평가합니다. 마치 계산기를 이용해 직접 손수 계산하지 않아도 되는 것처럼 말이죠.

하지만 주의도 필요합니다. 인공지능의 관심도가 높아진 만큼 많은 매체에서 다양한 이야기를 접할 것입니다. 실질적으로 잘 설명해주는 사람도 있지만, 공상 스토리를 만들어서 빨리 따라가지 않으면 뒤처진다고 허황된 이야기를 하는 사람도 많습니다.

처음 휴대전화가 등장했을 때, 사람들은 주로 개인 스케줄을 관리하는 어시스턴트로서의 서비스가 킬러 앱*이 되어 큰 역할을 할 것이라고 예상했습니다. 그러나 지금 우리는 언제 어디서나 영상을 보고, SNS를 즐기며 소통할 수 있다는 점에서 휴대전화의 큰 가치를 느끼고 있습니다.

인공지능의 경우도 지금은 혼란스러운 상황이지만 진짜 우리 생활을 달라지게 할 수 있는 구체적이고 확실한 서비스가 나온다면 정리되지 않을까 조심스럽게 예상합니다. 지금까지는 마치 골드러시 시대*처럼 청바지, 텐트, 샵 등을 만드는 과정 중에 있는 것 같습니다.

그러니 독자 분들도 인공지능을 열정적이지만 신중하게 이해하고 판단해 좋은 목적으로 사용할 수 있는 현명한 사람이 되시길 간곡히 부탁드립니다. 인공지능이 본인에게 도움이 된다면 적극적으로 사용하는 것이 삶의 질을 높이는 데 크게 도움이 되리라 생각합니다.

*킬러 앱

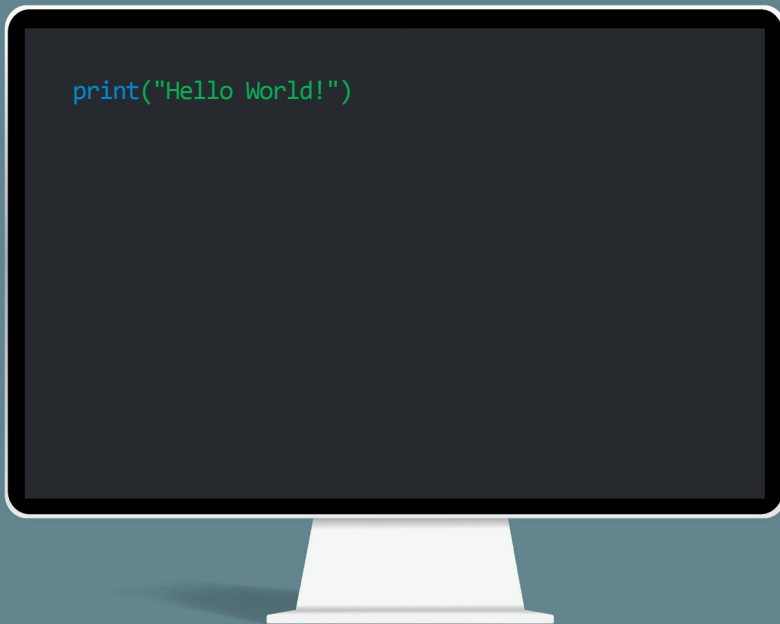
시장에 등장하자마자 다른 경쟁 제품을 몰아내고 시장을 완전히 재편할 정도로 인기를 누리는 상품이나 서비스.

*골드러시 시대

19세기 캘리포니아에서 금이 발견되면서 1850년대의 미국 개척민들이 캘리포니아로 몰려간 현상.

첫 코딩은 계획대로 될 거야!

에디터 저니



인공지능 개발의 세계에 들어온 모두를 환영한다. 지금까지 인공지능에 대한 흥미로운 이야기를 많이 들었다면, 이 글의 단계를 함께 밟으며 직접 인공지능을 구현해보자.

인공지능의 구현 과정은 사실 요리와 비슷하다.

요리를 시작하기 위해서는 자신에게 딱 맞는 주방을 골라야 한다. 주방의 형태나 특징이 본인에게 잘 어우러지는지는 중요한 요소이다. 이처럼 인공지능을 구현하기 위해 먼저 자신에 맞는 개발 환경을 선택한다. 어떤 개발환경이 있는지 알아보고 자신의 상황에 맞는 선택을 해보자.

요리를 하기 위해 식재료는 당연히 필요하다. 맛있는 음식을 만들려면 좋은 식재료가 필요하듯이 성능이 우수한 인공지능을 구현하려면 우수한 데이터를 수집하는 과정은 필수적이다. 어떻게 데이터를 얻을 수 있는지 알아보자.

중식에는 웍이 필요하고 한식에는 뚝배기가 필요하듯이, 어떤 음식을 만드는데 따라 필요한 조리 도구가 다르다. 이처럼 똑같은 프로그래밍 언어를 사용하더라도 인공지능을 만들기 위해서는 알맞은 프레임워크가 필요하다. 그중 PyTorch(파이토치)가 무엇인지 이해해보자.

위의 모든 준비를 마쳤다고 해도 처음 요리를 하는 사람이라면 시작하기 어려운 것이 사실이다. 그래서 우리는 요리책이나 동영상 보며 참고도 하고 따라하며 요리 실력을 기른다. 인공지능 구현도 처음에는 막막할 수 있다. 그래서 함께 공유하고 공부할 수 있는 개발자를 위한 사이트를 참고할 수 있다. 어떤 사이트가 있는지 둘러보고, 적극적으로 사용해보자.

이제부터, 위의 내용을 자세히 공부해보자.

개발환경 선택

데이터 수집

프레임워크
활용

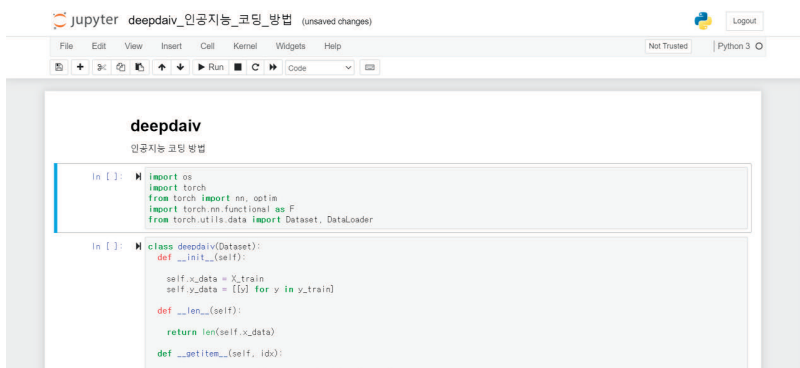
개발 참고
사이트

Stage 1

: 개발환경 선택

인공지능에서는 프로그래밍 언어로 대개 파이썬을 이용한다. 파이썬으로 코드를 작성하고 편집하고 저장할 수 있는 여러 종류의 프로그래밍 툴_{Tool}이 있다. 어떤 종류의 에디터가 있는지 알아보고 나에게 맞는 것을 찾아보자.

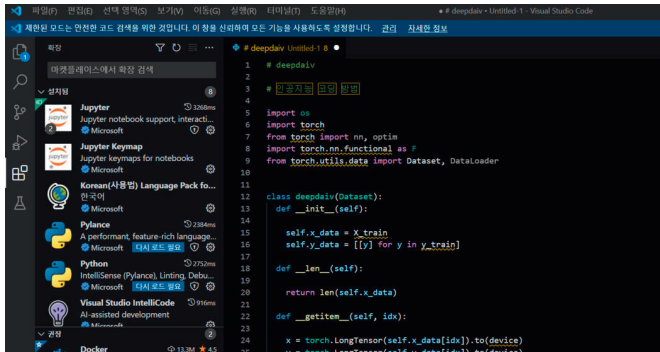
1.1. Jupyter Notebook



- 장점**
1. 코드를 작은 블록인 셀 _{Cell} 단위로 작성하고 실행할 수 있다. 셀 단위의 코드를 즉각적으로 실행해 결과를 볼 수 있는 대화식 _{Interactive} 구조이기에 빠르고 간편하게 수정, 테스트할 수 있다.
 2. 마크다운 _{Markdown} 기능으로 코드를 문서로 만들 수 있어 코드에 대한 설명과 정리가 쉽다.
 3. 파이썬 뿐 아니라 다른 다양한 프로그래밍 언어를 지원해서 작업을 용이하게 할 수 있다.
 4. Jupyter Notebook 파일은 .ipynb 확장자로 저장되는데, 이는 다른 사람들과 쉽게 공유가 가능하고 GitHub이나 Google Colab와도 호환성이 우수하다.

- 단점**
1. 셀과 셀 사이의 관계를 찾기 힘들어 작성된 다른 코드를 이해하기 힘들다. 코드를 셀 단위로 적는 것이 오류를 잡기 편할 수 있지만, 복잡한 코드에서는 셀 간의 상호작용을 관찰하기 어려울 수 있다.
 2. Jupyter Notebook으로 만들어진 코드는 하나의 큰 파일로 저장되기 때문에 버전 관리가 어렵다. 이는 함께 하는 프로젝트를 하는 협업 과정에서 어려움을 초래할 수 있다.

1.2. Visual Studio Code



- 장점**
1. 다양한 Extension을 이용할 수 있어 다양한 프로그램 언어나 효율적인 기능 지원을 추가해 편리하게 활용할 수 있다.
필요한 Extension은 그림과 같이 검색을 통해 설치하고 이용이 가능하다.
 2. 파이썬 외에도 거의 모든 프로그래밍 언어를 지원하고 이를 사용할 수 있다.
 3. 사용자가 쉽게 커스터마이징 할 수 있어서 자신에게 딱 맞는 환경을 구축할 수 있다.
 4. 내장된 터미널을 제공하기 때문에 이를 이용해서 코드 편집 창 내에서 명령어를 실행할 수 있어 빠르고 간편하다.

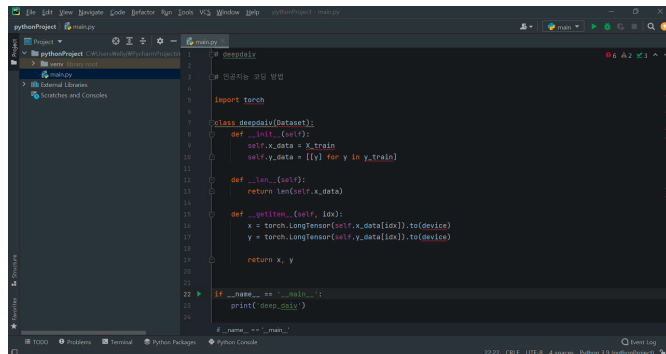
- 단점**
1. 빠른 연산 및 학습을 위해 GPU가 필요하다.
 2. 다양한 창을 열고 작업할 수 있는데, 인터페이스가 복잡해 보일 수 있다. 익숙하지 않다면 디버깅 과정이 불편할 수 있다.
 3. 대규모 프로젝트를 관리하는 경우에 어려움을 겪을 수 있다.

Q. 빠른 연산을 위해서 왜 GPU가 필요한 건가요?

CPU는 컴퓨터의 중앙 처리 장치이고, GPU는 복잡해지는 그래픽을 처리하기 위한 장치로 등장했다. GPU가 발전함에 따라 그래픽 처리 능력도 향상되었을 뿐 아니라 다른 계산 역시 용이해졌다.

GPU와 CPU는 각각 다른 방식으로 연산을 처리한다. CPU는 순차적으로 복잡한 작업을 빠르게 처리한다. 반면에 GPU는 동시에 많은 간단한 작업을 병렬로 처리한다. 딥러닝에서는 대량의 수치 연산을 동시에 수행하기 때문에 GPU의 병렬 처리 능력이 장점으로 작용한다.

1.3. PyCharm



- 단점**
1. 무료 버전도 있지만, 일부 기능은 유료 버전으로만 지원이 가능하다.
 2. 인터페이스를 처음 사용하면 복잡하게 느껴질 수 있다.
 3. 라이브러리 설치와 같은 초기 설정 과정에서 어려움을 느낄 수 있다.
 4. 많은 리소스를 요구하기 때문에 CPU가 상당 수준 필요할 수 있고, 빠른 연산 및 학습을 위해서는 역시나 GPU가 필요하다.

1.4. Google Colab(Colaboratory)



- 장점**
1. 웹 브라우저를 사용하여 개발하기 때문에 언제 어디서든 개발하든 내용을 불러올 수 있다.
 2. 클라우드에서 CPU와 GPU를 제공하기 때문에 고가의 CPU와 GPU를 마련할 필요가 없다.
 3. GitHub 등과 의 연동을 통해 자유롭게 소스를 올리고 가져올 수 있는 개발 환경을 제공한다.
 4. 구글 드라이브로 공유할 수 있어서 다수가 동시에 편집하는 협업 작업에 유리하다.

- 단점**
1. 무료 버전도 있지만, 일부 기능은 유료 버전으로만 지원이 가능하다.
 2. 인터페이스를 처음 사용하면 복잡하게 느껴질 수 있다.
 3. 라이브러리 설치와 같은 초기 설정 과정에서 어려움을 느낄 수 있다.
 4. 많은 리소스를 요구하기 때문에 CPU가 상당 수준 필요할 수 있고, 빠른 연산 및 학습을 위해서는 역시나 GPU가 필요하다.



내게 필요한 개발 환경은?

Jupyter Notebook

내용을 깔끔하게 정리하고 각 입출력 결과를 바로바로 확인하고 싶은 코딩 입문자에게 추천!

PyCharm

파이썬을 중점으로 코드 구조를 쉽게 파악하고 싶은 사람에게 추천!

Visual Studio Code

다양한 확장팩을 추가하여 자신에게 맞는 환경을 구축하고 싶은 사람에게 추천!

Google Colab

개발 환경 구축이 쉬워 초심자와 딥러닝을 실행하기 위한 GPU가 필요한 사람에게 추천!

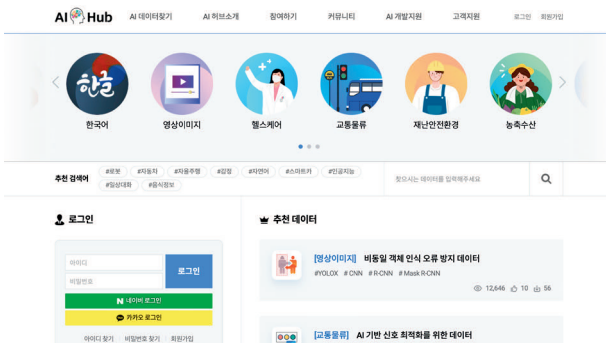
누구나 본인에게 어울리거나 본인이 좋아하는 스타일이 있지 않은가?

자신에게 필요한 개발 환경을 찾아 활용한다면 더욱 개성 넘치고 즐거운 인공지능 코딩이 될 것이라 기대한다.

Stage 2: 데이터 수집

인공지능을 개발하기 위해 중요한 작업 중 하나는 데이터를 수집하는 것이다. 데이터의 종류나 질이 인공지능 모델의 성능을 크게 좌우하기 때문이다. 그렇다면 인공지능 개발을 위한 데이터는 어디서 어떻게 구할 수 있을지 대표적인 사이트를 중심으로 알아보자.

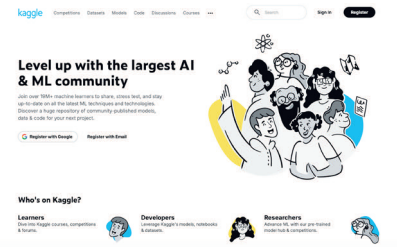
2.1. AI Hub



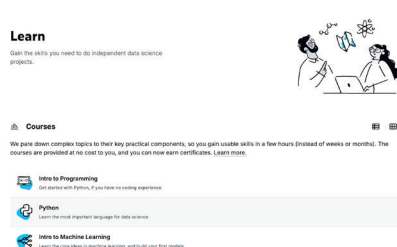
AI Hub는 한국지능정보사회진흥원(NIA)에서 운영하는 플랫폼으로 인공지능 구현에 필요한 데이터를 제공한다. 텍스트, 음성, 이미지, 영상 등 형태도 다양하며 금융, 농축 수산, 안전, 관광 등 분야도 다양해 필요한 데이터를 얻기 쉽다. 이는 주기적으로 갱신되는 최신의 고품질 한국 데이터라는 크나큰 장점을 가지고 있다.

데이터를 찾다보면, 한국어 데이터나 우리나라 문화 또는 사회에 맞는 데이터를 찾는 것이 쉬운 일이 아니다. 그럴 때 AI Hub를 이용한다면 도움이 될 수 있다. 국가 차원에서 관리해 문제 없는 품질이 검증된 데이터이기 때문이다. 하지만 데이터를 사용하기 위해 승인을 받아야 할 수 있고, 대용량 데이터이기 때문에 고성능 컴퓨터에서 활용을 권장한다.

2.2. Kaggle



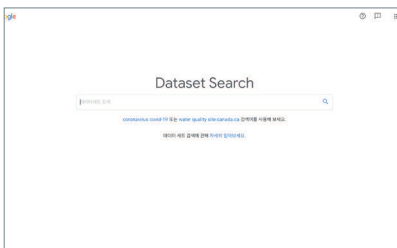
Kaggle은 전 세계의 많은 사람들이 이용하는 플랫폼으로, 방대한 양의 데이터를 무료로 구할 수 있다. 본인이 원하는 종류의 데이터는 Kaggle에서 검색과 전처리 과정을 통해 얻을 수 있을 것이다. 또한 커뮤니티나 협업을 통해 쉽게 배우고 공유를 할 수 있어서 유용하게 사용할 수 있다. Kaggle 경진대회에서 다양한 머신러닝의 도전과제를 확인해볼 수도 있다.



교육용 자료 또한 잘 정리되어 있다. Kaggle Learn에서 단계별로 쉽게 학습할 수 있는 예제나 코드를 통해 빠르게 학습할 수 있는 자료도 있어 추가적인 공부를 원하면 사용할 수 있다.

책 뒤쪽에 있는 <deep daiv into Kaggle & Dacon> 아티클을 참고하면 Kaggle에 대한 이해도와 활용도가 더욱 높아질 것이다.

2.3. Google Data Search



키워드 검색을 활용하면 Google에서 다양한 데이터셋을 쉽게 찾아볼 수 있다. 방대한 데이터 속에서 필요한 정보를 효과적으로 필터링하고 얻을 수 있어 매우 유용하다. 또한, 특정 플랫폼에 제한되지 않고 간단한 검색만으로 손쉽게 접근할 수 있어 탐색 과정이 효율적이다.

Google과 비슷한 인터페이스를 가지고 있어, 처음 이용해도 검색에 큰 무리가 없다.



추가로, 아래 사이트에서도 많은 데이터를 얻을 수 있으니 목적에 맞게 여러 사이트를 활용하길 바란다.

- UCI Machine Learning Repository
- UN data
- 서울 열린 데이터 광장

Stage 3: 프레임워크 활용

어떤 개발 환경에서 어떤 데이터를 이용해 인공지능을 만들지 선택했다면, 이제 실제로 코드를 짜야 할 시간이다. 다만, 인공지능을 구현하기 위해서는 일반적인 파이썬 환경에 특화된 프레임워크를 추가하면 더욱 효과적으로 활용할 수 있다. 이 중에서도 특히 많이 사용되는 대표적인 인공지능 프레임워크인 PyTorch(파이토치)를 중심으로 살펴보자.

PyTorch

PyTorch란?

PyTorch는 페이스북 인공지능 연구팀이 딥러닝의 구현을 위해 개발한 파이썬 기반의 오픈소스 머신러닝 라이브러리이다. 2016년 9월 처음으로 공개됐으며 개발자와 연구자들이 GPU를 이용해 쉽게 인공신경망 모델을 만들고 학습시킬 수 있도록 도와준다.

PyTorch는 왜 사랑받는가?

1. PyTorch는 딥러닝을 사용할 수 있도록 많은 내장 모델을 제공한다.
2. PyTorch는 Define-by-Run 방식으로, 연산 정의와 값의 초기화가 동시에 이루어지는 방식을 사용한다. 그래서 값을 저장할 공간을 따로 만들 필요가 없어서 모델 그래프를 데이터에 따라 자유롭게 조절할 수 있다는 점에서 편리하다.
3. PyTorch는 직관적이고 간결한 코드로 사용자가 점점 늘어나고 있고, 이에 따라 많은 인공지능 코드를 이해할 수 있다.
4. PyTorch는 파이썬의 비슷한 문법 체계 내에서 개발이 되었기 때문에 파이썬의 기능을 쉽게 사용할 수 있다.

부록: PyTorch vs. TensorFlow

TensorFlow는 Define-and-Run 방식으로, 먼저 모델을 만들고 난 뒤에 값을 따로 넣어주는 방식을 사용한다. 이에 반해 PyTorch는 Define-by-Run 방식으로, 그래프를 먼저 만들어 놓고 값을 넣는 것이 아니기 때문에 코드의 길이가 상대적으로 짧아지고 코드 이해가 쉬워진다.

현재는 TensorFlow에 비해 PyTorch는 머신러닝 분야에서 세계적인 주목을 받으며 꾸준히 점유율을 높이고 있어, 공부에 활용할 수 있는 자료 역시 빠른 속도로 늘어나고 있다. 또 공통으로 파이썬을 사용하긴 하지만 PyTorch가 조금 더 간결한 문법을 사용하기 때문에 비교적 진입장벽이 낮고, 사용할 수 있는 모델의 개수가 많다는 장점이 있다.

PyTorch의 원리

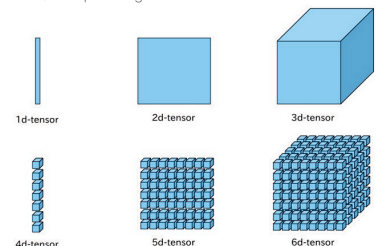
PyTorch에서 사용되는 가장 핵심적인 개념은 텐서 Tensor 이다. 텐서는 다차원의 배열을 이용해 데이터를 표현하기 위해 사용하는 개념이다.

파이썬에서 Numpy와, 수학에서 matrix와 비슷한 개념으로 차원이 커지는 데이터를 다루기 위해 사용된다.

하지만 차이점이 있다면, Numpy와 다르게 GPU를 이용해 가속할 수 있고 자동 미분이 가능해 딥러닝 학습에 효율적이다. 또한 Matrix는 2차원 배열로 행렬 연산에 사용되지만, 텐서는 더욱 고차원 배열을 포함할 수 있어 복잡한 데이터 표현에 효과적이다.

그리고 PyTorch에서 텐서를 사용해 데이터를 쉽게 조작할 수 있다. 텐서를 생성하고, 모양을 바꾸거나, 덧셈과 곱셈 같은 연산을 수행할 수 있고, 브로드캐스팅 Broadcasting 을 통해 다양한 크기의 텐서도 함께 계산할 수 있으며, 인덱싱 Indexing 과 슬라이싱 Slicing 으로 필요한 부분만 추출할 수 있다. 이런 다양한 기능을 활용해 데이터 처리와 모델 학습이 훨씬 더 유연하고 간편하다.

출처: opeoplearning-lab



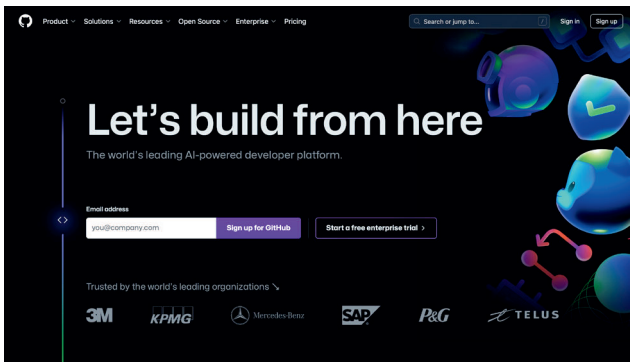
Stage 4: 개발자를 위한 사이트 참고

이제 PyTorch를 이용하고 데이터까지 구했다면 어떤 모델을 어떻게 학습시킬지 선택하고, 학습 방법에 맞는 자신의 개발 환경을 구축해야 한다. 하지만 처음부터 어떤 모델이나 학습 방법을 써야 하는지 아이디어를 떠올리기는 쉽지 않다.

또한 다수가 함께 코드를 짜야 하는 경우도 있는데, 이런 경우에는 어떻게 해야 프로젝트를 원활하게 진행할 수 있을까?

이러한 고충을 해결해 줄, 개발자에게 도움이 되는 몇 가지 사이트를 소개한다.

4.1. GitHub



GitHub(깃허브)란, Git(깃)이라는 버전 관리 시스템으로 관리하는 프로젝트를 올려놓는 사이트이다.

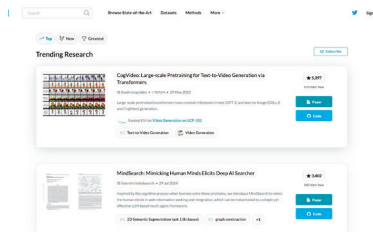


다수가 함께 코드를 수정하고 저장할 하다 보면 어느 것이 가장 최근 것인지, 혹은 이전 버전에서 더 추가해야 할 사항이 있을 때 어느 버전인지 알기 어렵다. 그래서 버전을 관리하는 시스템이 Git이다. 하지만 Git은 로컬 저장소에서만 작동하기 때문에 이를 협업에 이용하기 위해서는 모두가 볼 수 있도록 온라인상에 공유해야 한다.

이렇게 Git으로 관리한 프로젝트를 호스팅하고 올려놓는 사이트가 GitHub인데, 블로그처럼 개인의 프로필을 꾸미고 프로젝트를 올릴 수 있다.

GitHub에서는 개인의 프로젝트를 올려두고 함께 협업할 수 있을 뿐만 아니라, 다양한 개발자의 오픈 소스를 확인하고 서로 지식을 공유하고 확인하는 창구로 사용되기도 한다.

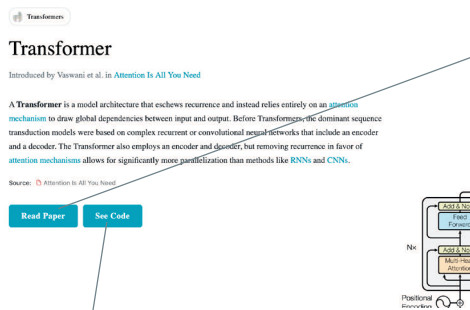
4.2. Papers with Code



Papers with Code는 이름에서 알 수 있듯이, 논문과 관련 코드를 한곳에서 확인할 수 있는 서비스이다. 논문의 구현 방식을 살펴볼 수 있도록 논문과 함께 코드가 제공되며, 해당 코드는 GitHub에 연결되어 있어 쉽게 접근할 수 있다. 이론, 개념과 더불어 코드까지 함께 살핀다면 논문에 대한 이해도가 높아질 것이다.

에디터 저니와 함께 사용해보는 Paper with Code 와 GitHub

많이 언급되는 Transformer 모델에 대해 알아보자.

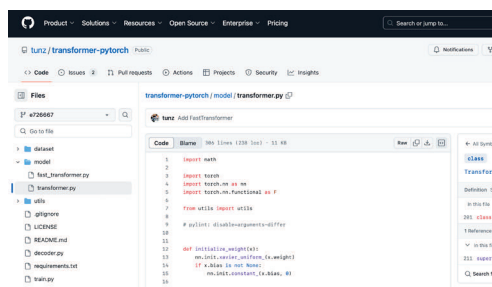


(1) Read Paper



'Read Paper' 버튼을 클릭하면 transformer의 논문인 'Attention Is All You Need' 논문을 확인할 수 있다. 이를 통해 논문을 직접 읽으며 모델에 대한 이해를 할 수 있다.

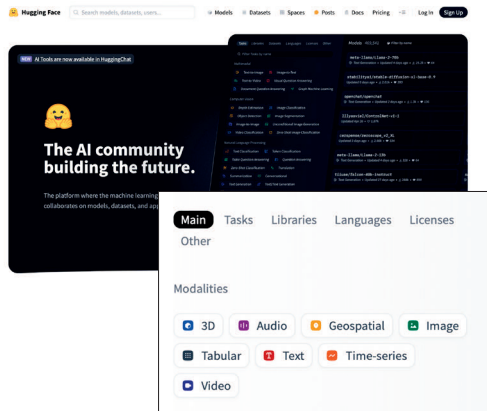
(2) See Code



'See Code' 버튼을 클릭하면 GitHub와 연동되어 Transformer에 대한 코드를 볼 수 있다. 논문을 통해 얻은 이론적 개념을 코드로 다시금 확인해보며 해당 모델에 대한 심도있는 이해를 할 수 있다.

코드를 직접 Colab으로 구현하면서 하이퍼파라미터 등을 튜닝해보는 것도 실력 향상에 도움이 될 것이다.

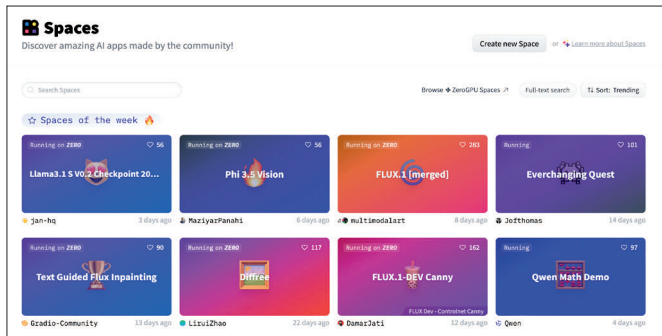
4.3. Hugging Face



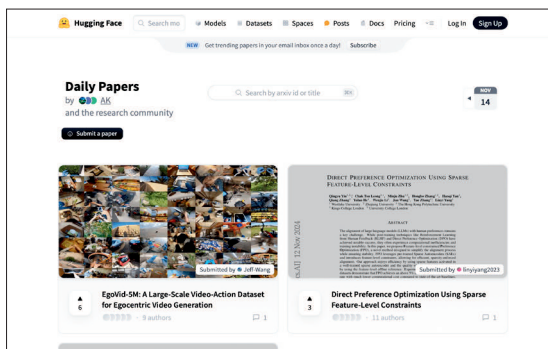
Hugging Face는 자연어 처리 및 머신러닝 분야에서 많이 사용되는 플랫폼이다. 특정 태스크에 대한 데이터를 얻을 수 있을 뿐 아니라 다양한 모델을 확인해 볼 수 있다. 사전 학습된 모델을 이용할 수 있어 인공지능 개발에 유용하게 쓰인다.

자세히 알아보자면, 모델 허브에서는 다양한 모델을 업로드하고, 쉽게 공유하며, 오픈소스로 공개할 수 있다. 또한 파이프라인(Pipeline)을 제공해 모델을 손쉽게 불러와 사용할 수 있도록 하여, 복잡한 설정 없이도 쉽게 시작할 수 있다.

데이터셋 허브에서는 방대한 양의 대규모 데이터셋을 얻을 수 있다. 다양한 분야의 데이터를 검색해 모델 학습에 필요한 데이터를 쉽게 사용할 수 있다.

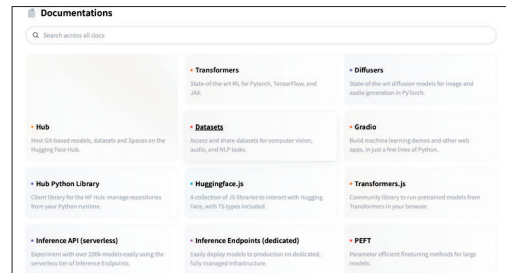


Hugging Face Spaces도 존재한다. 이는 사용자가 만든 머신러닝 모델이나 프로젝트를 웹 애플리케이션 형태로 개발하여 배포하고 공유하는 사이트이다. 간단한 웹 애플리케이션을 올릴 수 있으며, 이를 통해 모델과 상호작용할 수 있다. 이를 통해 보다 편하고 활동적으로 다른 사람의 프로젝트를 맛보고 공유할 수 있다.



Daily Papers라는 플랫폼에서 인공지능 분야의 최신 연구 논문을 매일 소개받을 수도 있다. 인공지능 개발 동향을 파악할 수 있고, 최신 트렌드에 쉽게 접근할 수 있어서 인공지능에 대한 흥미를 얻기 좋다.

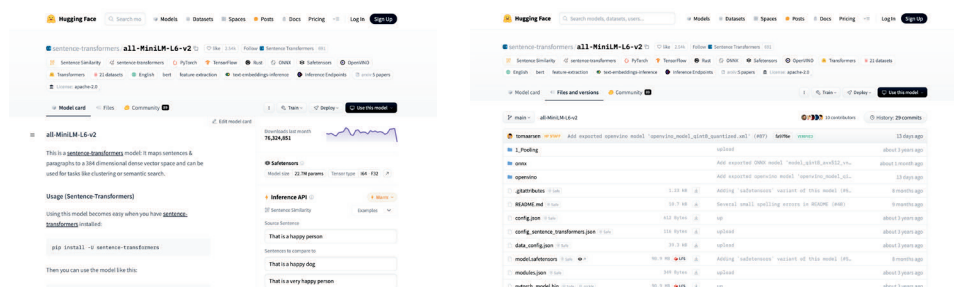
활발한 커뮤니티가 운영되며, 정리가 잘 된 문서도 존재하여 입문자부터 전문가까지 모두 함께 이용할 수 있다는 장점도 있다.



에디터 저니와 함께 사용해보는 Hugging Face

마찬가지로 Transformer에 대해 찾아보도록 하자. Transformer에 대해 검색을 하면 많은 모델과 데이터가 뜨는 것을 확인할 수 있다.

그 중 sentence-transformers에 대한 내용을 구체적으로 보면,

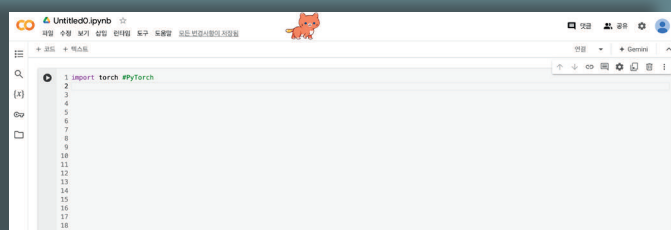


어떻게 라이브러리를 사용할 수 있는지 알 수 있고, GitHub와 마찬가지로 repository를 확인할 수 있다. sentence-transformers에 사용한 데이터셋도 찾아보고, 이를 통해 직접 학습을 진행할 수 있다.

마치며...

자 이제 여러분의 차례다.

빈 도화지를 채워나가는 화가처럼, 빈칸을 여러분의 코드로 채우다 보면 새로운 세상에 눈뜨게 될 것이다.



참고 자료 / 출처

Part.1 인공지능에 뛰어들 준비

인공지능 입문을 위한 필수 개념 총정리 작성자: 저니

곽종호. (2024). 인공지능 개론. 복랩.

김수빈. (2022). 파이썬으로 맛보는 인공지능 한 조각. 디지털북스.

박해선. (2020). 혼자 공부하는 머신러닝+딥러닝 (구글 코랩으로 환경 설정 없이 실습 가능). 한빛미디어.

이재구. (2020, August 27). 규칙기반 AI vs 머신러닝... 최적의 활용 조건은?. AI 타임즈. <https://www.aitimes.com/news/articleView.html?idxno=129476>

Doosan Encyclopedia. (n.d.). 강화 학습. <https://terms.naver.com/entry.naver?docId=6713957&cid=40942&categoryId=32845>

Doosan Encyclopedia. (n.d.). 약한 인공지능과 강한 인공지능. <https://terms.naver.com/entry.naver?docId=6704179&cid=40942&categoryId=32845>

freeCodeCamp. (2017, August 12). Chihuahua or muffin? My search for the best computer vision API. <https://www.freecodecamp.org/news/chihuahua-or-muffin-my-search-for-the-best-computer-vision-api-cbda4d6b425d/>

Google for Developers. (n.d.). 클러스터링이란 무엇일까요? <https://developers.google.com/machine-learning/clustering/overview?hl=ko>

MathWorks. (n.d.). Convolutional neural network란? <https://kr.mathworks.com/discovery/convolutional-neural-network.html>

Naver Dictionary. (2024, July 23). 라이브러리. <https://ko.dict.naver.com/#/entry/koko/79c47b89d17a4ca2ae60af25baf0f44e>

Naver Dictionary. (n.d.). 과적합. <https://terms.naver.com/entry.naver?docId=6653423&cid=69974&categoryId=69974>

Naver Knowledge Encyclopedia. (n.d.). 손실 함수. <https://terms.naver.com/entry.naver?docId=6653622&cid=69974&categoryId=69974>

Naver Knowledge Encyclopedia. (n.d.). t-SNE. <https://terms.naver.com/entry.naver?docId=6653602&cid=69974&categoryId=69974>

Rukshan Pramoditha. (2021, December 26). The concept of artificial neurons (perceptrons) in neural networks. Medium. <https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc>

Ted. (2020, October 5). RNN(Recurrent Neural Network, 순환신경망)을 이해해보자. Medium. <https://medium.com/humanscape-tech/rnn-recurrent-neural-network-순환신경망-을-이해해보자-1697a5472af2>

YOLOv3: An incremental improvement. (2018, August). Joseph Redmon, Ali Farhadi. <https://arxiv.org/abs/1804.02767>

Wikipedia. (n.d.). File:C Logo.png. https://commons.wikimedia.org/wiki/File:C_Logo.png

Wikipedia. (n.d.). File:Python logo and wordmark.svg. https://commons.wikimedia.org/wiki/File:Python_logo_and_wordmark.svg

Wikipedia. (n.d.). 파일:PyTorch logo black.svg. https://ko.wikipedia.org/wiki/파일:PyTorch_logo_black.svg

AI와 함께 하는 답다의 하루 작성자: 한나

- 김지엽. (2023, 4월 6일). [논문리뷰] Training language models to follow instructions with human feedback. 전생했더니 인공지능이었던 건에 대하여. Retrieved December 1, 2024, from <https://kimjy99.github.io/%EB%85%B0%EB%AC%B8%EB%A6%AC%EB%B7%B0/instructgpt/>
- 손엄지. (2023, 12월 18일). 안 읽은 카톡방, AI로 요약해 준다. 뉴스1. Retrieved December 1, 2024, from <https://www.news1.kr/it-science/general-it/5264761>
- 손현. (2024, 3월 29일). 토스증권, 개인투자자를 위한 콘텐츠 표준을 만든다. 토스피드. Retrieved December 1, 2024, from <https://blog.toss.im/article/outsight-for-retail-investor>
- 안준영. (2022, 11월 5일). Deepfake의 원리: 당신의 예상보다 훨씬 쉽고, 대단한 기술. Justin4AI. Retrieved December 1, 2024, from <https://velog.io/@hewas1230/deepfake-principle>
- 알바트로스. (2024, 3월 31일). GPT-3.5 해부하기: 인간 피드백에 의한 강화학습(RLHF)으로 더욱 똑똑해진 인공지능. 브런치. Retrieved December 1, 2024, from <https://brunch.co.kr/@harryban0917/290>
- 카카오. (2023, 12월). AI도 편해야 카톡이지! 카톡설명서. Retrieved December 1, 2024, from <https://talktips.kakao.com/bridge/67>
- 카카오뱅크 금융기술연구소. (2023, 12월 20일). 카카오뱅크의 금융특화 언어모델 KF-DeBERTa 개발기. Retrieved December 1, 2024, from <https://ai.kakaobank.com/f982c5b8-9cbd-4ef1-8ebd-d7359a70284b>
- 토스. (2023, 6월 9일). 토스 | SLASH 23 - 머신러닝으로 더 똑똑하게 증권 뉴스 제공하기 [Video]. YouTube. <https://www.youtube.com/watch?v=dnxaTrKJr0c>
- 한채림. (2023, 10월 6일). 연구 답보이스. cozyrim.log. Retrieved December 1, 2024, from <https://velog.io/@cozyrim/%EC%97%B0%EA%B5%AC-%EB%94%A5%EB%B3%B4%EC%9D%B4%EC%8A%A4>
- Anthropic. (2024, June 21). Introducing Claude 3.5 Sonnet. Retrieved December 1, 2024, from <https://www.anthropic.com/news/claude-3-5-sonnet>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 1, 4171–4186. <https://doi.org/10.48550/arXiv.1810.04805>
- LETR. (2023, 3월 24일). 누가 누가 잘하나_(1) 텍스트 요약 비교. LETR works. Retrieved December 1, 2024, from <https://www.lettr.ai/ko/blog/tech-230324>
- OpenAI. (2022, January 27). Aligning language models to follow instructions. OpenAI. Retrieved December 1, 2024, from <https://openai.com/index/instruction-following/>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Ziegler, D. M. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730–27744. <https://doi.org/10.48550/arXiv.2203.02155>
- Pino. (2023, 5월 9일). Text-To-Speech 정리. Pino's Blog. Retrieved December 1, 2024, from <https://pinopino.tistory.com/entry/Text-To-Speech-%EC%A0%95%EB%A6%AC>
- Tolosana, R., Vera-Rodríguez, R., Fierrez, J., Morales, A., & Ortega-García, J. (2020). Deep learning for deepfakes creation and detection: A survey. arXiv preprint arXiv:2004.12247. <https://doi.org/10.48550/arXiv.2004.12247>
- Wikidocs. (n.d.). 22-01 어텐션을 이용한 텍스트 요약. 딥 러닝을 이용한 자연어 처리 입문. Retrieved December 1, 2024, from <https://wikidocs.net/72820>

지역별 AI 정책 톨아보기 작성자: AI Future Thinkers

- 김나리. (2024, 9월 3일). [답페이크 쇼크] 방치됐던 'AI 기본법' 국가인공지능안 출범으로 탄력받나. 이투데이. <https://www.ETtoday.co.kr/news/view/2396775>
- 배문규. (2024, 7월 7일). 다시 시작된 'AI 기본법' 논의... '고위험' '처벌' 어떻게 담을까. 경향신문. <https://www.khan.co.kr/economy/economy-general/article/202407071550001>
- 박강민, 장진철, 안성원. (2024). 유럽연합 인공지능법(EU AI Act)의 주요내용 및 시사점. 소프트웨어정책연구소, 이슈리포트. 2024.07.
- 박지민. (2024, 11월 25일). 한국, AI 보안 특허출원 세계 3위... 삼성전자 6위. 조선일보. Retrieved December 1, 2024, from <https://www.chosun.com/economy/science/2024/11/24/XP62H2VYPFCOVBDABO4I6VYOG4/>
- 법무법인(유) 화우. (2024, 4월). 생성형AI의 국제 규제 동향: 미국, EU, 중국을 중심으로 한 규제 동향 훑어보기. Retrieved December 1, 2024, from https://www.hwawoo.com/newsletter/2024_04_11/240411_k_a.pdf
- 설동훈. (2024, 9월 1일). [이슈분석] EU 고강력 AI 규제 나서자...기술 기업들 집단 반발. 뉴스드림. Retrieved December 1, 2024, from <https://www.newsdream.kr/news/articleView.html?idxno=66670>
- 오유빈. (2024). 미국의 인공지능 입법 현황과 바이든 행정부의 행정명령. 국회도서관. 현안 - 외국에선?. 2024-1
- 이경탁. (2024, 11월 7일). [트럼프 귀환] 미국 AI 행정명령, 새 행정부 하에서 어떻게 달라질까? 조선비즈. Retrieved December 1, 2024, from <https://biz.chosun.com/it-science/ict/2024/11/07/KYVZF5QGEBGX3GYLIDIIVO2FEE/>
- 이정현. (2024, 5월 16일). "한국, AI-사물인터넷-빅데이터 분석 도입률 OECD 1위". 연합뉴스. Retrieved December 1, 2024, from <https://www.yna.co.kr/view/AKR20240516045300017>
- 이정아. (2024). 미국의 인공지능(AI) 정책 전략 현황과 변화 방향: AI 지배력 강화와 초강대국 유지를 위해 미국은 어떻게 변화하고 있는가?. 한국지능정보사회진흥원. AI Report. 2024-3
- 이해원. (2024). 유럽연합 인공지능법: 주요 내용과 시사점. STARTUP ALLIANCE. 이슈페이퍼. 2024-04
- 홍완식. (2002). 유럽연합의 입법체계. 유럽연합정부론, 65-88.
- Al Jazeera. (2024, March 25). EU to investigate Apple, Google, Meta under new digital law. Al Jazeera. Retrieved December 1, 2024, from <https://www.aljazeera.com/economy/2024/3/25/eu-to-investigate-apple-google-meta-under-new-digital-law>
- Brussels Times. (2024, March 25). Apple, Google, Meta: New EU rules on internet gatekeepers take effect today. Brussels Times. Retrieved December 1, 2024, from <https://www.brusselstimes.com/955313/apple-google-meta-new-eu-rules-on-internet-gatekeepers-take-effect-today>
- European Parliament. (2023, June 8). EU AI Act: first regulation on artificial intelligence, Retrieved December 1, 2024, from <https://www.europarl.europa.eu/topics/en/article/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence>
- Federal Register. (2023, October 30). Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence. The White House. Retrieved December 1, 2024, from <https://www.whitehouse.gov/briefing-room/presidential-actions/2023/10/30/executive-order-on-the-safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence/>
- 법무법인(유) 화우. (2024, 4월). 생성형AI의 국제 규제 동향: 미국, EU, 중국을 중심으로 한 규제 동향 훑어보기. Retrieved December 1, 2024, from https://www.hwawoo.com/newsletter/2024_04_11/240411_k_a.pdf
- 설동훈. (2024, 9월 1일). [이슈분석] EU 고강력 AI 규제 나서자...기술 기업들 집단 반발. 뉴스드림. Retrieved December 1, 2024, from <https://www.newsdream.kr/news/articleView.html?idxno=66670>
- 오유빈. (2024). 미국의 인공지능 입법 현황과 바이든 행정부의 행정명령. 국회도서관. 현안 - 외국에선?. 2024-1
- 이경탁. (2024, 11월 7일). [트럼프 귀환] 미국 AI 행정명령, 새 행정부 하에서 어떻게 달라질까? 조선비즈. Retrieved December 1, 2024, from <https://biz.chosun.com/it-science/ict/2024/11/07/KYVZF5QGEBGX3GYLIDIIVO2FEE/>
- 이정현. (2024, 5월 16일). "한국, AI-사물인터넷-빅데이터 분석 도입률 OECD 1위". 연합뉴스. Retrieved December 1, 2024, from <https://www.yna.co.kr/view/AKR20240516045300017>

이정아. (2024). 미국의 인공지능(AI) 정책 전략 현황과 변화 방향: AI 지배력 강화와 초강대국 유지를 위해 미국은 어떻게 변화하고 있는가?. 한국지능정보사회진흥원. AI Report. 2024-3

이해원. (2024). 유럽연합 인공지능법: 주요 내용과 시사점. STARTUP ALLIANCE. 이슈페이퍼. 2024-04

홍완식. (2002). 유럽연합의 입법체계. 유럽연합정부론, 65-88.

Al Jazeera. (2024, March 25). EU to investigate Apple, Google, Meta under new digital law. Al Jazeera. Retrieved December 1, 2024, from <https://www.aljazeera.com/economy/2024/3/25/eu-to-investigate-apple-google-meta-under-new-digital-law>

Brussels Times. (2024, March 25). Apple, Google, Meta: New EU rules on internet gatekeepers take effect today. Brussels Times. Retrieved December 1, 2024, from <https://www.brusselstimes.com/955313/apple-google-meta-new-eu-rules-on-internet-gatekeepers-take-effect-today>

European Parliament. (2023, June 8). EU AI Act: first regulation on artificial intelligence, Retrieved December 1, 2024, from <https://www.europarl.europa.eu/topics/en/article/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence>

Federal Register. (2023, October 30). Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence. The White House. Retrieved December 1, 2024, from <https://www.whitehouse.gov/briefing-room/presidential-actions/2023/10/30/executive-order-on-the-safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence/>

Part.2 인공지능 세계에서 헤엄치기

2024 AI Trend Keyword 작성자: 느리

Anthropic. (2024, May 21). Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>

Ahn, M., Dwibedi, D., Finn, C., Gonzalez Arenas, M., Gopalakrishnan, K., Hausman, K., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kirmani, S., Leal, I., Lee, E., Levine, S., Lu, Y., Maddineni, S., Rao, K., Sadigh, D., Sanketi, P., Sermanet, P., Vuong, Q., Welker, S., Xia, F., Xiao, T., Xu, P., Xu, S., & Xu, Z. (2024). AutoRT: Embodied foundation models for large-scale orchestration of robotic agents. arXiv. <https://arxiv.org/abs/2401.12963>

Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., & Florence, P. (2023). PaLM-E: An embodied multimodal language model. arXiv. <https://arxiv.org/abs/2303.03378>

Figure. (2024). Figure 02. Retrieved December 1, 2024, from <https://www.figure.ai/>

Figure. (2024). Figure Status Update - OpenAI Speech-to-Speech Reasoning [Video]. YouTube. <https://www.youtube.com/watch?v=Sq1QZB5baNw&t=6s>

Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.

Lundberg, S. (n.d.). SHAP [Image]. GitHub. Retrieved from December 1, 2024, from <https://github.com/shap/shap>

Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., & Wei, F. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv preprint arXiv:2402.17764.

NVIDIA. (2023). NVIDIA ACE enhanced with dynamic responses for virtual characters [Video]. YouTube. <https://www.youtube.com/watch?v=cSSSn10HgZA&t=37s>

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should i trust you?" Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 1135-1144).

Samsung. (n.d.). Galaxy AI is here. Retrieved December 1, 2024, from <https://www.samsung.com/sec/galaxy-ai/>

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35, 24824-24837.

Yevgen Chebotar, Tianhe Yu. (2023, July 28). RT-2: New model translates vision and language into action. Google DeepMind. <https://deepmind.google/discover/blog/rt-2-new-model-translates-vision-and-language-into-action/>

Part.3 인공지능에 빠져들기

첫 코딩은 계획대로 될 거야! 작성자: 저니

Deepbak (2024, 5월 23일). tensor와 아이들(scalar, vector, matrix). DeepLearning-Lab. <https://deeplearning-lab.com/uncategorized/tensor/>

JeanMichealLee (2019, January 7). 3. PyTorch VS TensorFlow. Dev. Retrieved December 1, 2024, from <https://dev-jm.tistory.com/4>

Ryan O'Connor (2021, December 14). PyTorch vs TensorFlow in 2023. AssemblyAI. Retrieved December 1, 2024, from <https://www.assemblyai.com/blog/pytorch-vs-tensorflow-in-2023/>

TensorFlow. (n.d.). TensorFlow tutorials (Korean). Retrieved December 1, 2024, from <https://tensorflowkorea.gitbooks.io/tensorflow-kr/content/g3doc/tutorials/>

TensorFlow blog. (n.d.). Retrieved December 1, 2024, from <https://tensorflow.blog/>

Wikipedia. (n.d.). PyTorch. Retrieved December 1, 2024, from <https://ko.wikipedia.org/wiki/PyTorch>

Wikipedia. (n.d.). TensorFlow. Retrieved December 1, 2024, from <https://en.wikipedia.org/wiki/TensorFlow>

OPINION: Apple intelligence는 최고가 될 수 있을까? 작성자: 정환

Apple. (n.d.). Apple intelligence. Retrieved December 1, 2024, from <https://www.apple.com/apple-intelligence/>

You, X., et al. (2024). Ferret-UI: Grounded mobile UI understanding with multimodal LLMs. arXiv preprint. <https://arxiv.org/pdf/2404.05719>

세상을 바꾸는 AI 기업들 작성자: 느리

삼성생명. (2023). [삼성생명 TVCF] 좋은 소식의 시작 [Video]. YouTube. <https://www.youtube.com/watch?>

Anthropic. (2024, June 21). Introducing Claude 3.5 Sonnet. Retrieved December 1, 2024, from <https://www.anthropic.com/news/claude-3-5-sonnet>

Anthropic. (2023, September 19). Anthropic Responsible Scaling Policy. Retrieved December 1, 2024, from <https://www.anthropic.com/news/anthropic-responsible-scaling-policy>

Boss, M., Huang, Z., Vasishtha, A., & Jampani, V. (2024). SF3D: Stable Fast 3D Mesh Reconstruction with UV-unwrapping and Illumination Disentanglement. arXiv preprint arXiv:2408.00653.

Deloitte. (2024, 2월 23일). 미래를 결정해야 하는 순간: 생성형AI 도입 현장에서 확인한 시사점. Retrieved from December 1, 2024, from <https://www2.deloitte.com/content/dam/Deloitte/kr/Documents/technology-media-telecommunications/2024/kr-all-The-state-of-Gen-AI-report.pdf>

Elevenlabs. (n.d.). Create a replica of your voice that sounds just like you. Retrieved from December 1, 2024, from <https://elevenlabs.io/voice-cloning>

ElevenLabs. (2024). ElevenLabs Iconic Voices - Judy Garland [Video]. YouTube. <https://www.youtube.com/watch?v=Zo5-rhYOINK&t=11s>

Heinz. (2022). Heinz A.I. Ketchup. [Video]. YouTube. <https://www.youtube.com/watch?v=LFmpVy6eGXs>

Midjourney (n.d.). Discord Interface. Retrieved from December 1, 2024, from <https://docs.midjourney.com/docs/midjourney-discord>

Mistral AI team. (2024, July 18). Mistral Nemo. Mistral AI. Retrieved December 1, 2024, from <https://mistral.ai/news/mistral-nemo/>

Philip Eu. (2024, 2월 15일) 가우디오 스튜디오 – 내 최애곡의 새로운 면모. 가우디오랩. Retrieved from December 1, 2024, from <https://www.gaudiolab.com/ko/blog/165>

Stability.ai. (2024, August 1). Introducing Stable Fast 3D: Rapid 3D Asset Generation From Single Images. Retrieved from December 1, 2024, from <https://stability.ai/news/introducing-stable-fast-3d>

Stability.ai. (n.d.). Stable Diffusion 3 Medium. Hugging Face. Retrieved from December 1, 2024, from <https://huggingface.co/stabilityai/stable-diffusion-3-medium>

Rio Oh. (2024, 7월 27일) ICML 논문 맛보기: A demand-driven perspective on Generative Audio AI. 가우디오랩. Retrieved from December 1, 2024, from <https://www.gaudiolab.com/ko/blog/148>

deep daiv into Kaggle & Dacon 작성자: 한나

Kaggle. (n.d.). How to Use Kaggle: Competitions. Retrieved from December 1, 2024, from <https://www.kaggle.com/docs/competitions>

편집인 이성배
기획총괄 최목원 김나영
글 강지현 김정환 김하늘 이성배 장한나 조유림
디자인 최목원 이성배

초판 1쇄 2024년 12월 10일
1차 수정 2024년 12월 10일

해당 창작물의 저작권은 다이브에게 있습니다.
무단전재 및 재배포를 금지합니다.